



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 34

Dr. Asad Mahmood

April 24, 2026

Lecture Outline

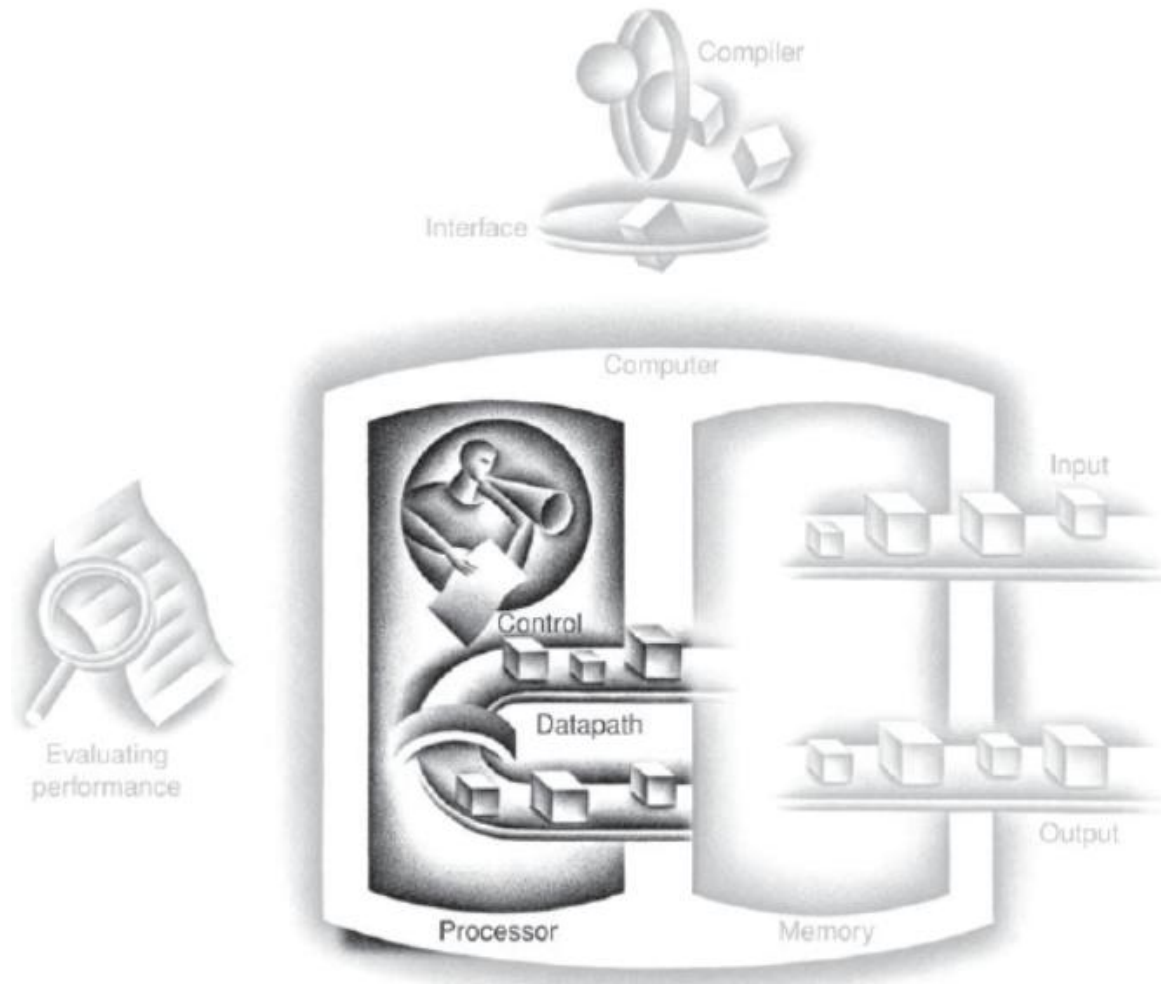
- Announcements (Assignment and Quiz 4)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.6 An Overview of Pipelining
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.7 Pipelined Datapath and Control

Chapter 4

The Processor

Chapter Theme

The Five Classic Components of a Computer



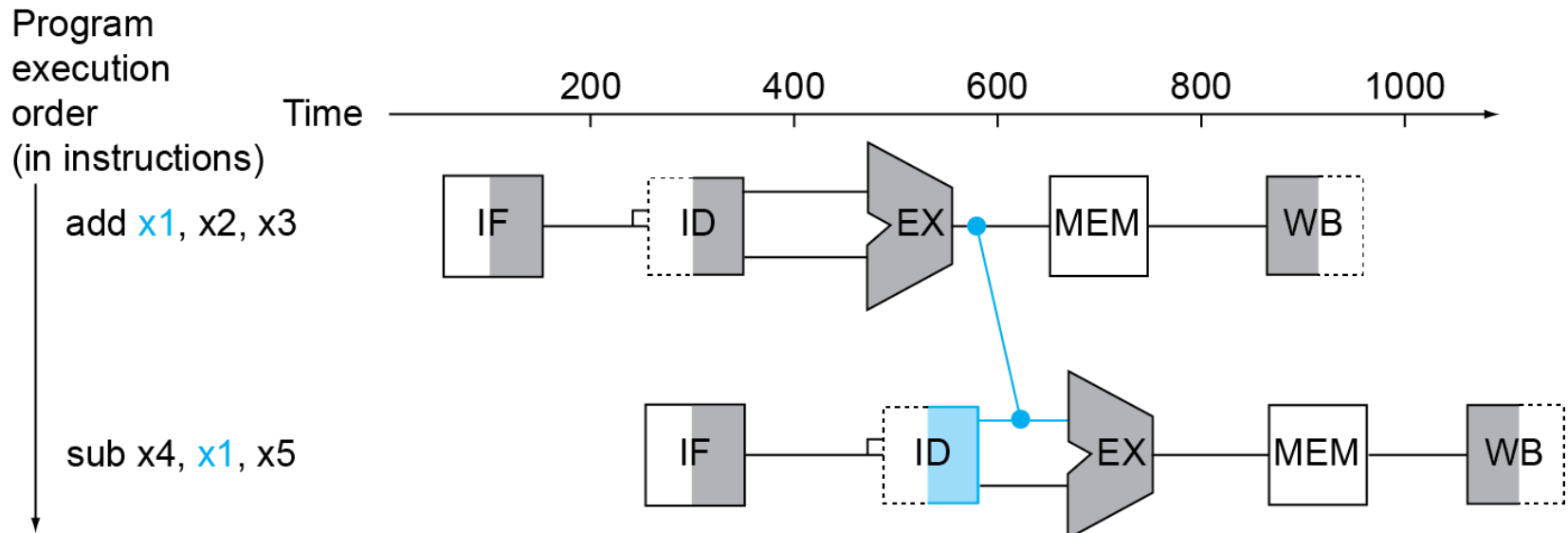
Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

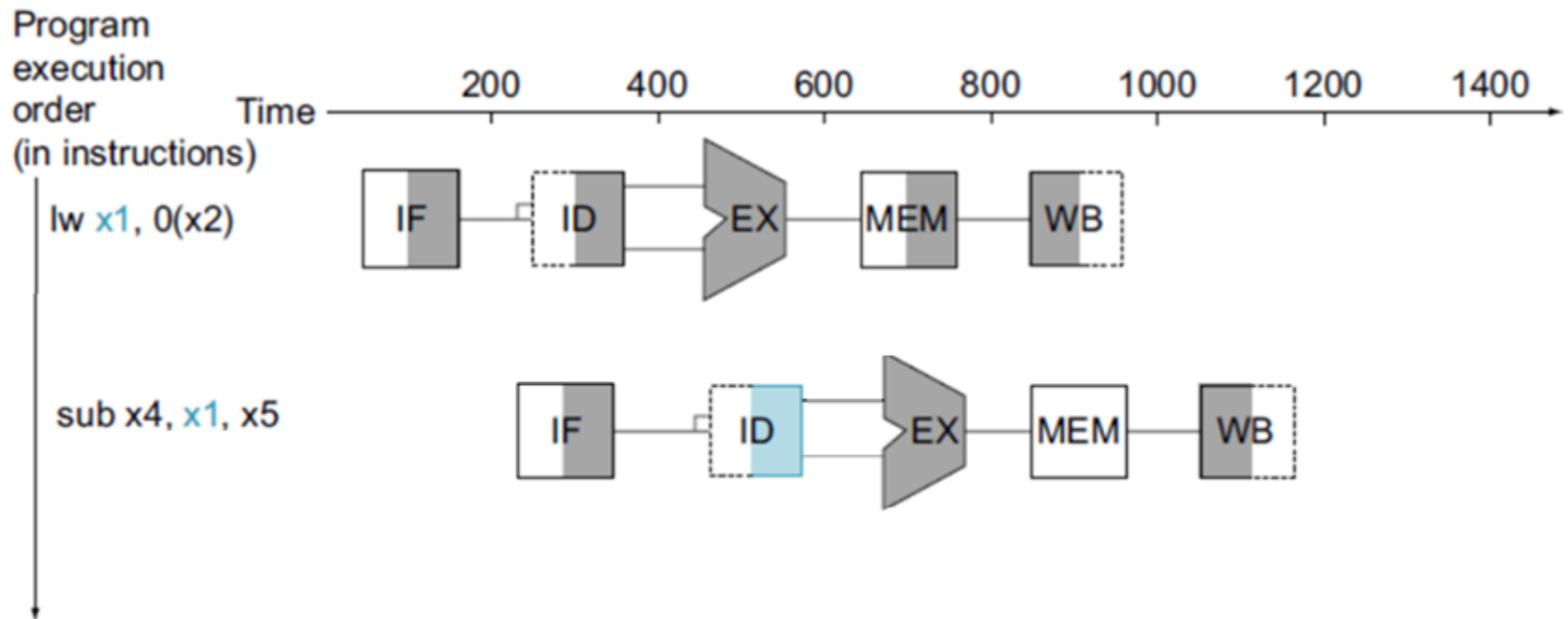
Forwarding (aka Bypassing)

- Use result as soon as it is computed
 - Requires extra connections in the datapath



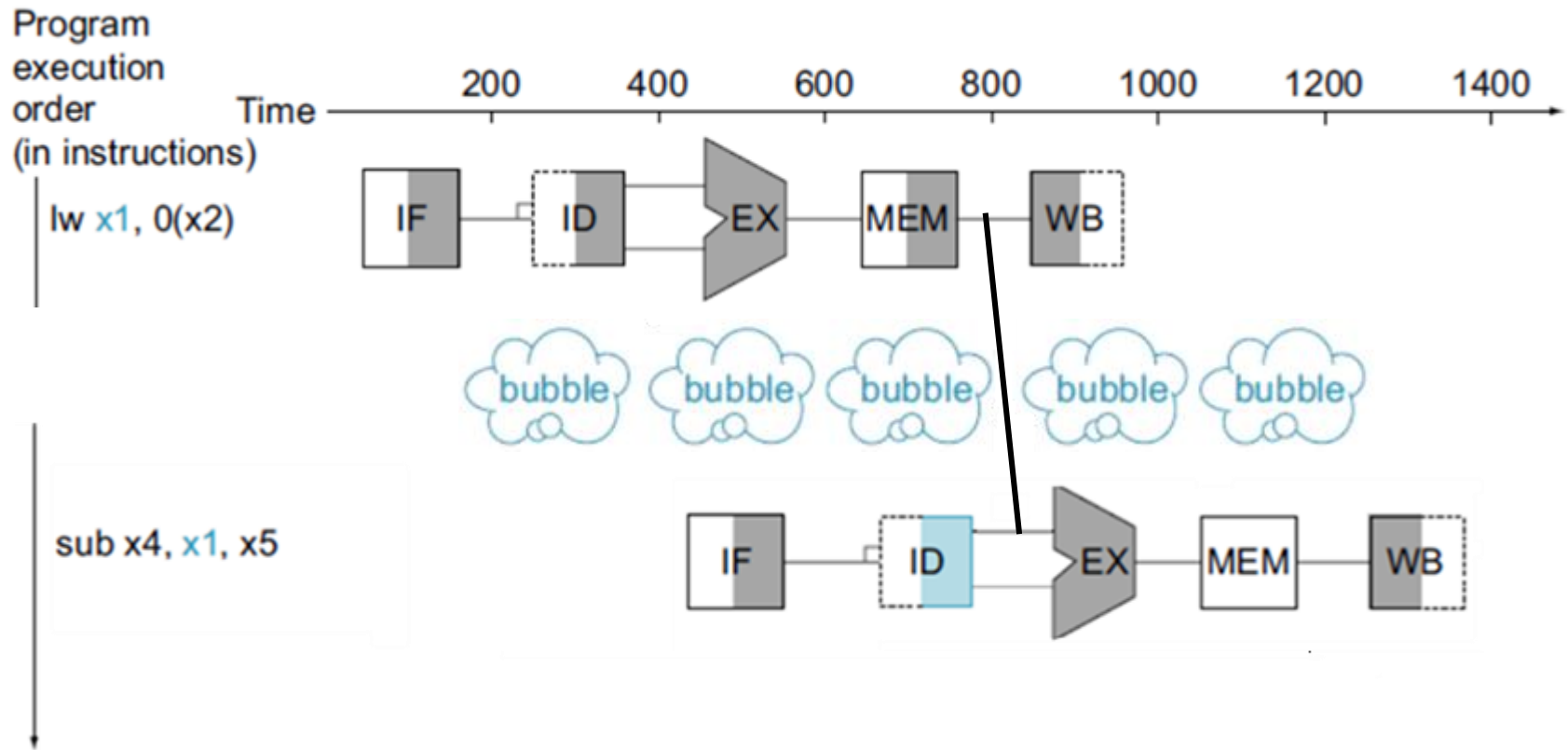
Load-Use Data Hazard

- Can't always avoid stalls by forwarding
 - If value not computed when needed
 - Need to stall! (Add a bubble)
 - Solution also possible via software re-ordering



Load-Use Data Hazard

- Can't always avoid stalls by forwarding



Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for $a = b + e; c = b + f;$
- Stalls?

lw x1, 0(x31)	// Load b
lw x2, 4(x31)	// Load e
add x3, x1, x2	// b + e
sw x3, 12(x31)	// Store a
lw x4, 8(x31)	// Load f
add x5, x1, x4	// b + f
sw x5, 16(x31)	// Store c

Total cycles ?

Code Scheduling to Avoid Stalls

- **Reorder code** to avoid use of load result in the next instruction
- C code for $a = b + e; c = b + f;$

```
lw x1, 0(x31)
lw x2, 4(x31)
lw x4, 8(x31)
add x3, x1, x2
sw x3, 12(x31)
add x5, x1, x4
sw x5, 16(x31)
```

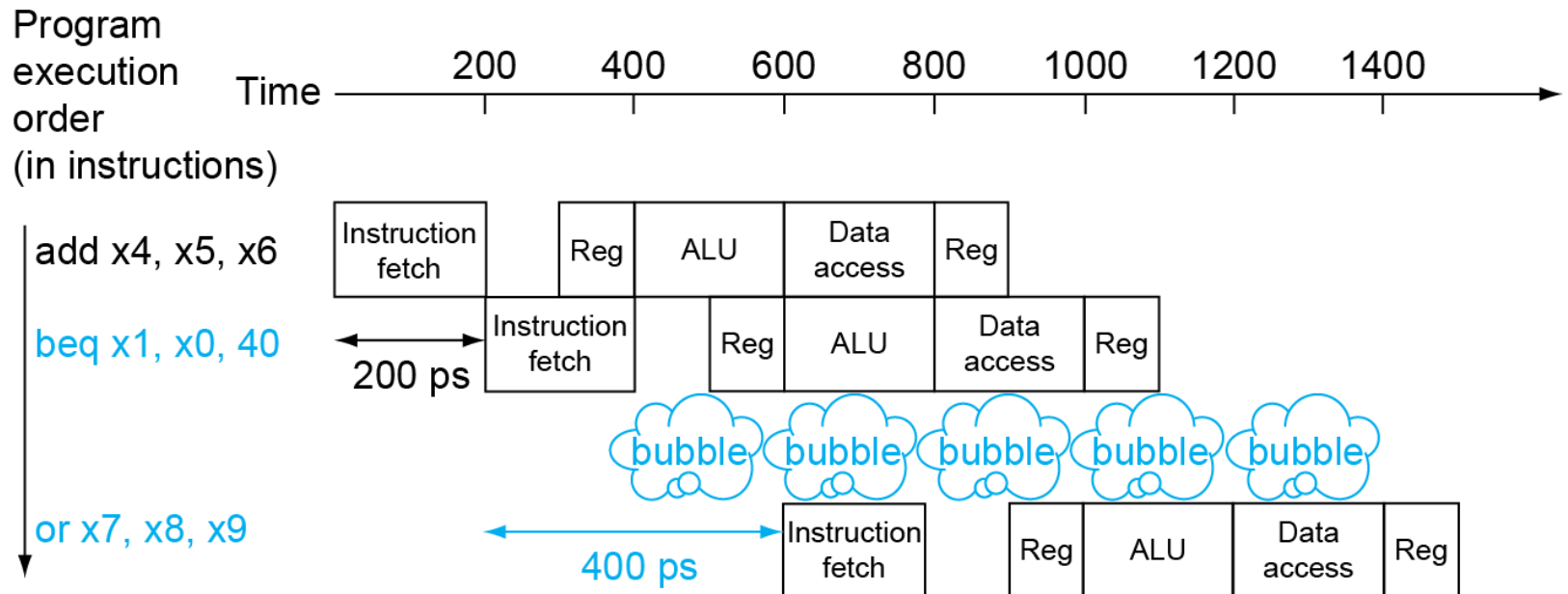
Total cycles

Control / Branch Hazards

- Branch determines flow of control
 - Fetching next instruction depends on branch outcome
 - Pipeline can't always know what the next instruction should be
- In RISC-V pipeline
 - Need to compare registers and compute target early in the pipeline
 - Add hardware to do it in ID stage

Stall on Branch

- Wait until branch outcome determined **before fetching next instruction**
 - Using **extra hardware**
- **Performance impact** (SPECint2006 with CPI = 1, 10% conditional branches, **impact?**)

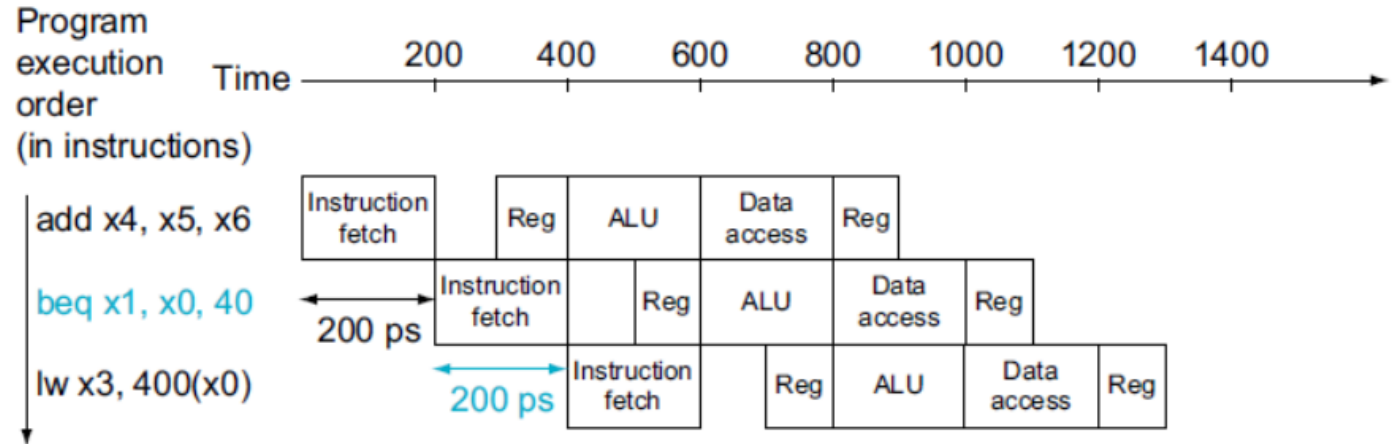


Branch Prediction

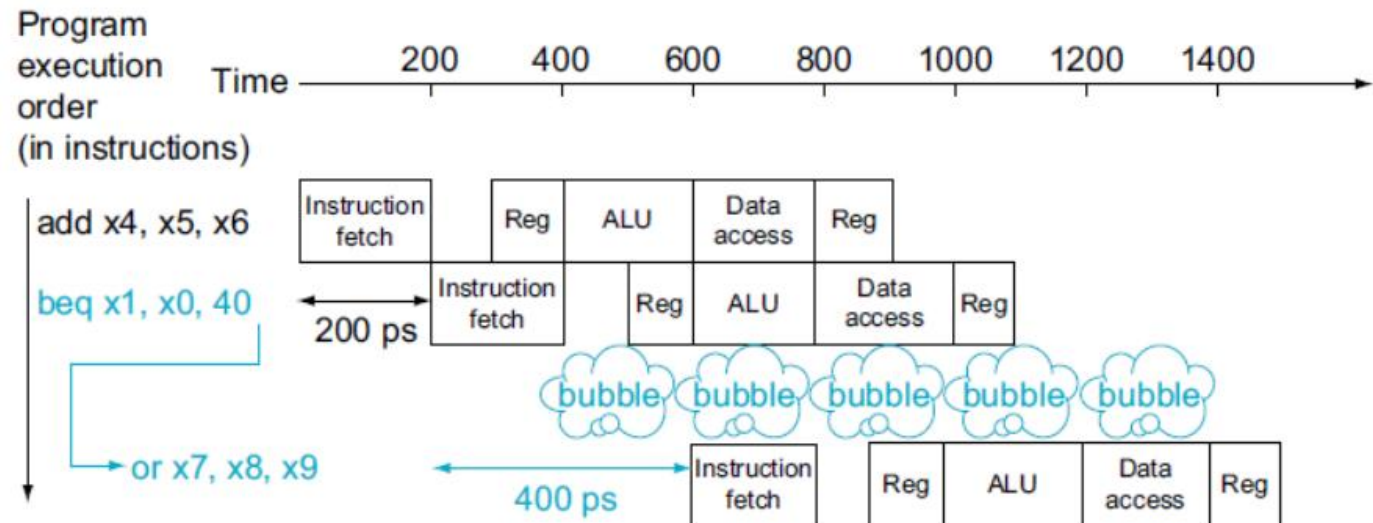
- Longer pipelines can't readily determine branch outcome early
 - Stall penalty becomes unacceptable
- Solution = Predict outcome of branch
 - Only stall if prediction is wrong
 - Why/How this may be a good option?
- In RISC-V pipeline
 - Can predict/assume branches not taken
 - Fetch instruction after branch, with no delay

Branch Prediction

Branch
not
taken
(default)



Branch
taken



More-Realistic Branch Prediction

- **Static** branch prediction
 - Based on **typical branch behavior**
 - Example: loop and if-statement branches
 - Predict **backward branches taken**
 - Predict forward branches not taken
- **Dynamic** branch prediction
 - Hardware measures **actual branch behavior**
 - e.g., **record recent history** of each branch
 - Assume future behavior will **continue the trend**
 - When wrong, stall while re-fetching, and update history

Pipeline Summary

- Pipelining improves performance by increasing instruction throughput
 - Executes multiple instructions in parallel
 - Each instruction has the same latency
- Subject to hazards
 - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

Chapter Outline

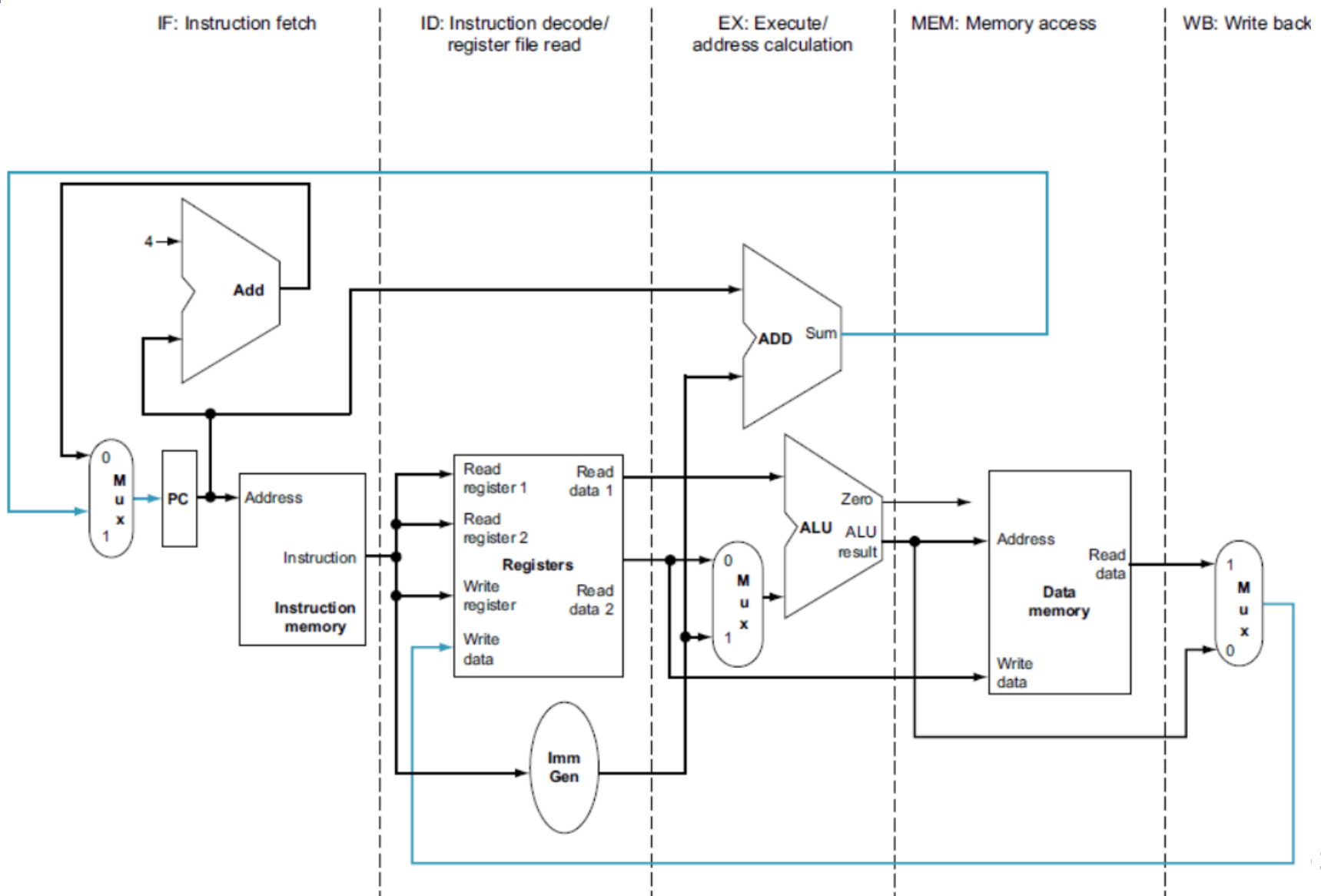
Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

RISC-V Pipelined Datapath

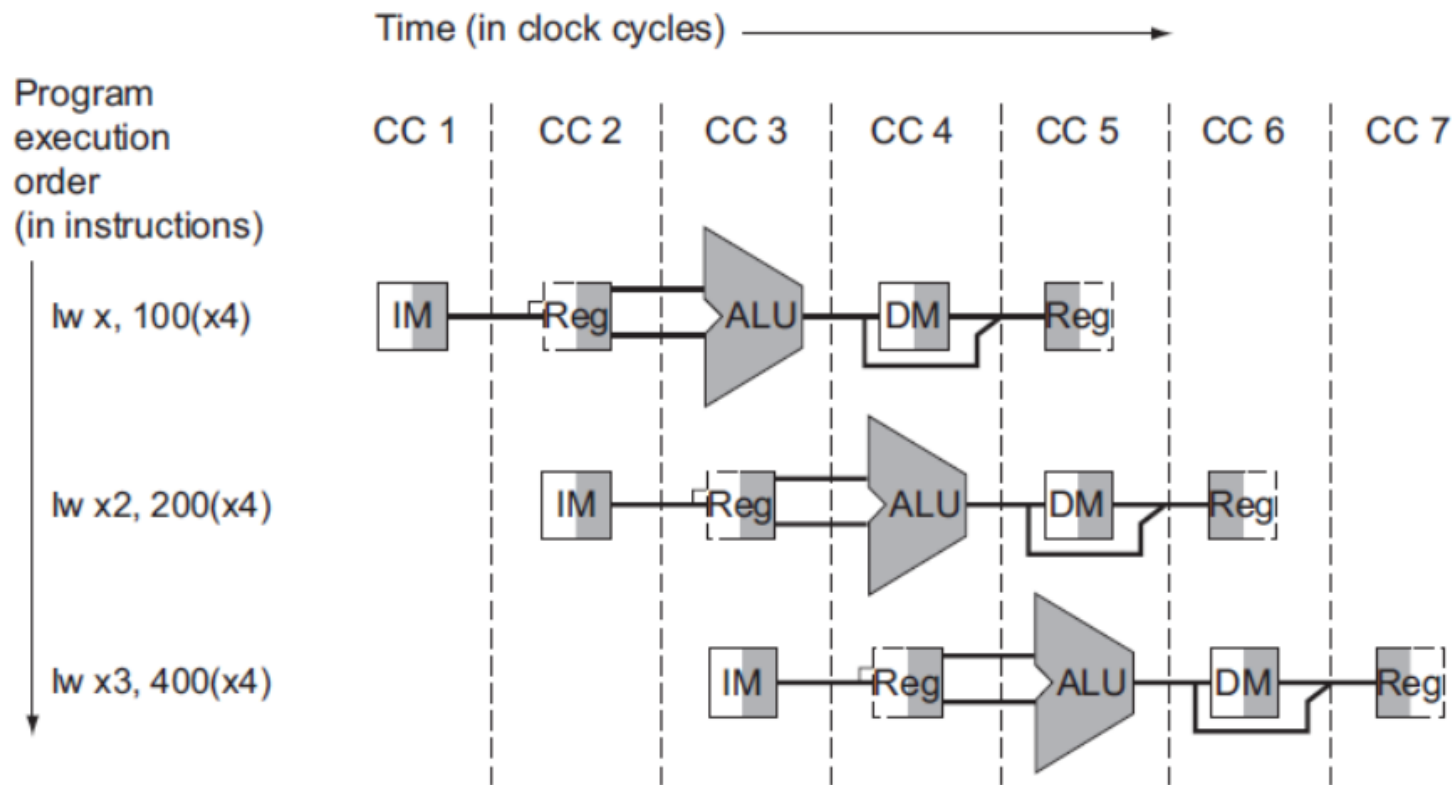
- We saw previously that instruction execution can be divided into 5 stages of pipeline
- How many instructions in parallel?
- RISC-V datapath can correspondingly be divided into 5 pieces/segments
- Data moves left-to-right in the datapath with two exceptions

RISC-V Single-cycle Datapath



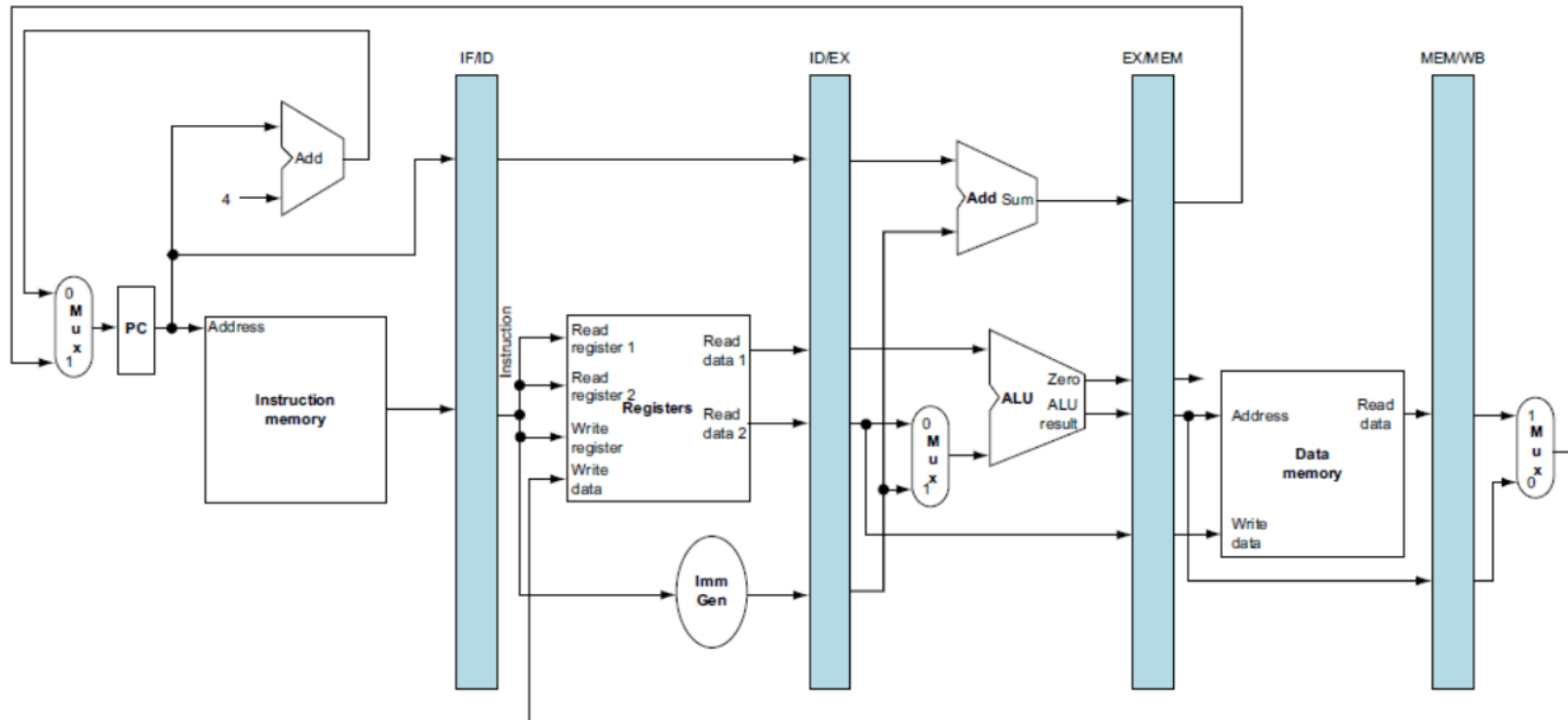
RISC-V Single-cycle Pipelined Datapath

- For ease of visualization, assume each instruction has its own dedicated datapath



Pipeline registers

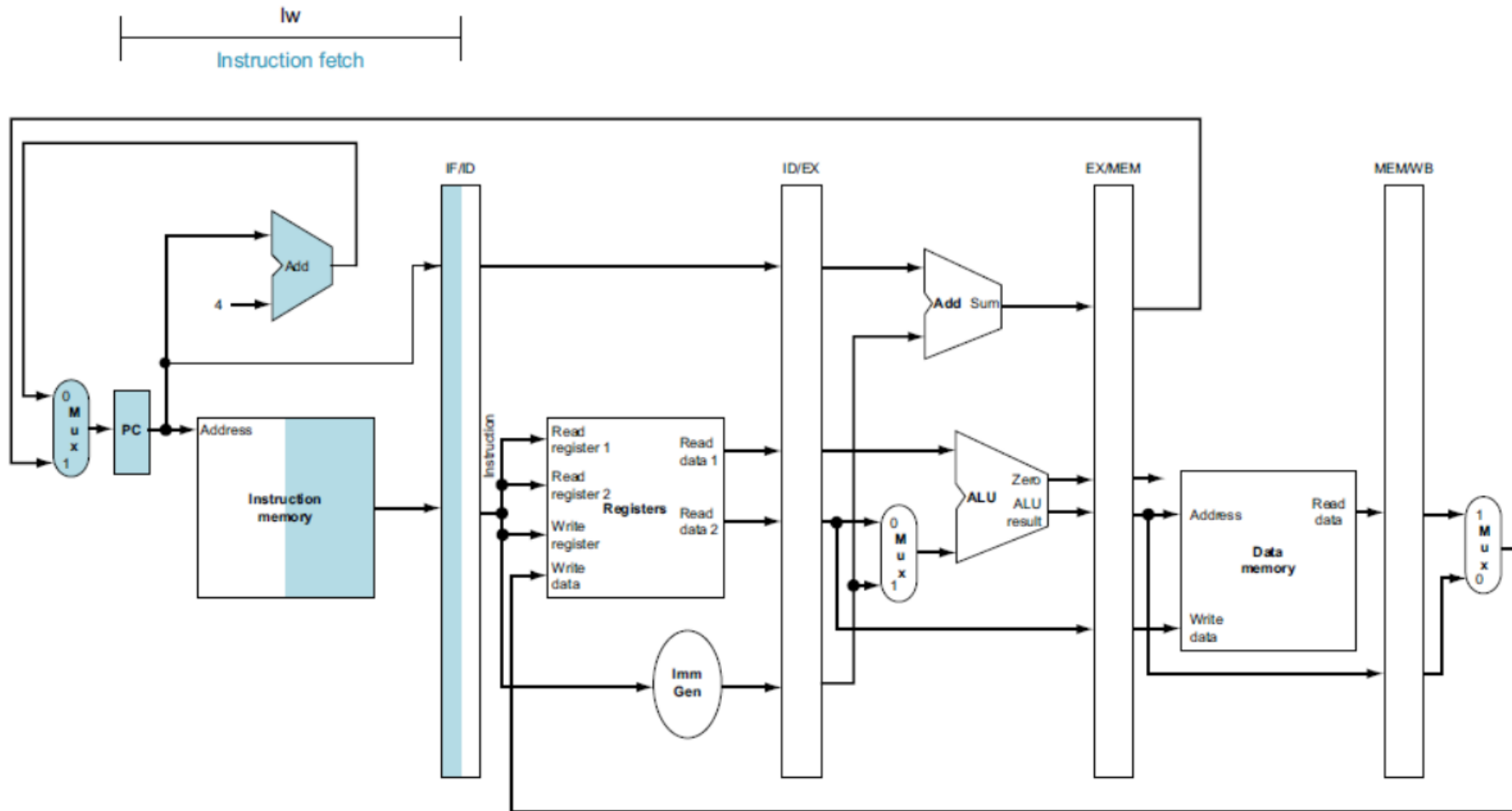
- In reality, a single datapath gets shared amongst instructions via use of registers
 - No pipeline register after the WB stage
 - Registers width?



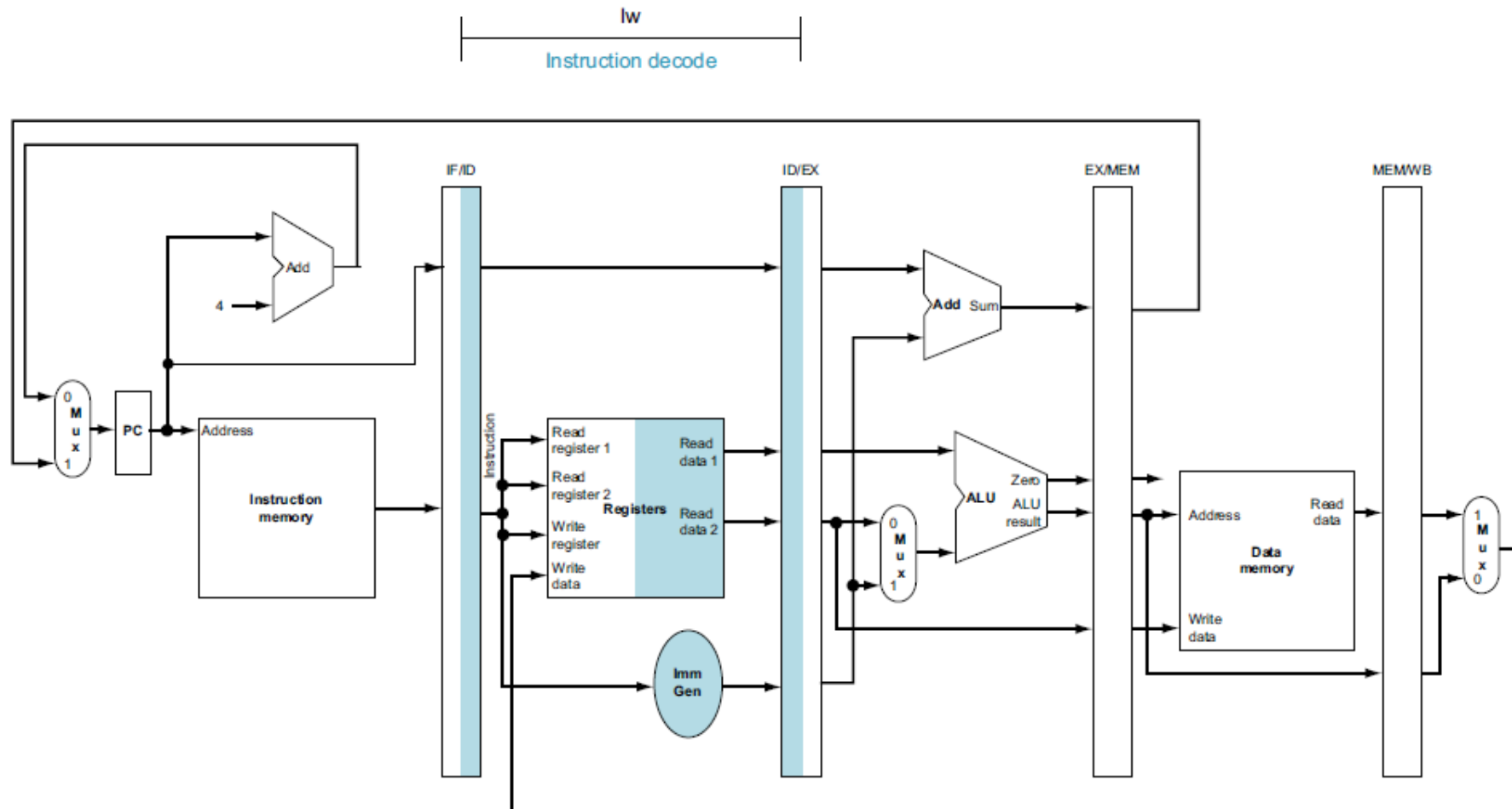
Pipeline Operation

- Pipeline operation will be demonstrated via cycle-by-cycle flow of instructions through the pipelined datapath. This will be done using two types of figures:
 1. “Single-clock-cycle” pipeline figures
 - Shows pipeline usage in a single cycle
 - Highlight resources used
 2. “Multi-clock-cycle” figures
 - Graph of operation over multiple clock cycles
- We will first look at “single-clock-cycle” diagrams for load & store

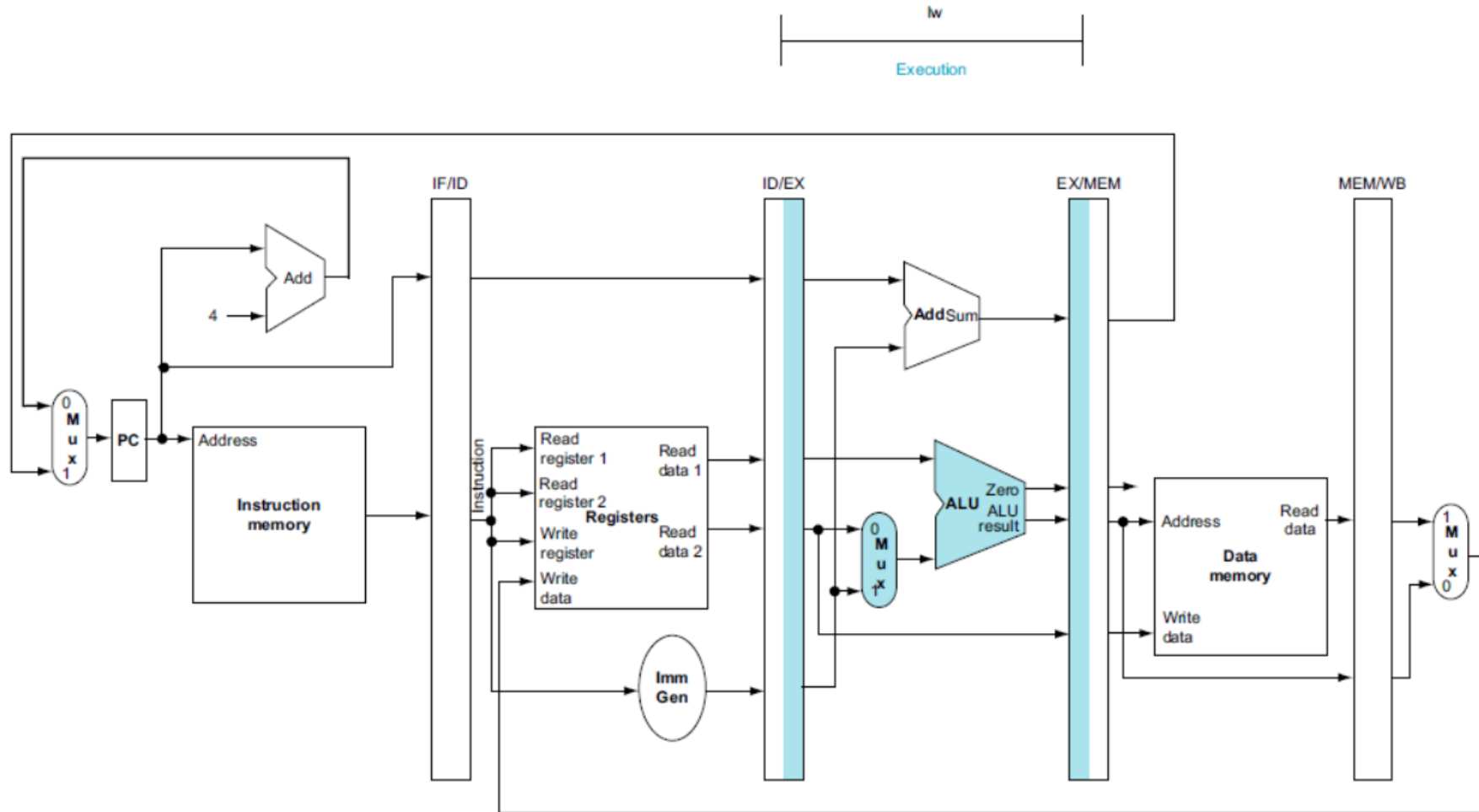
IF for Load, Store, ...



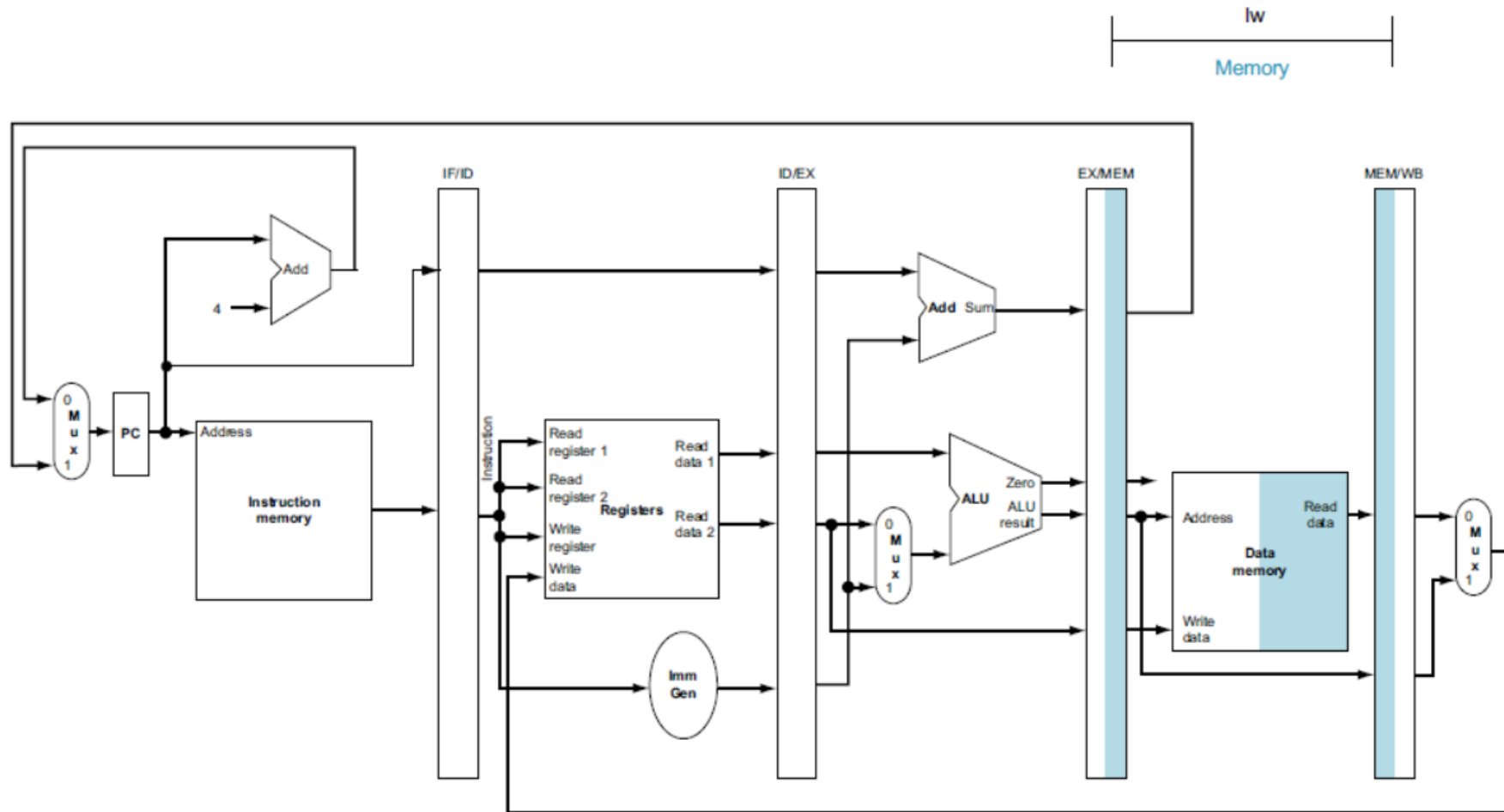
ID for Load, Store, ...



EX for Load

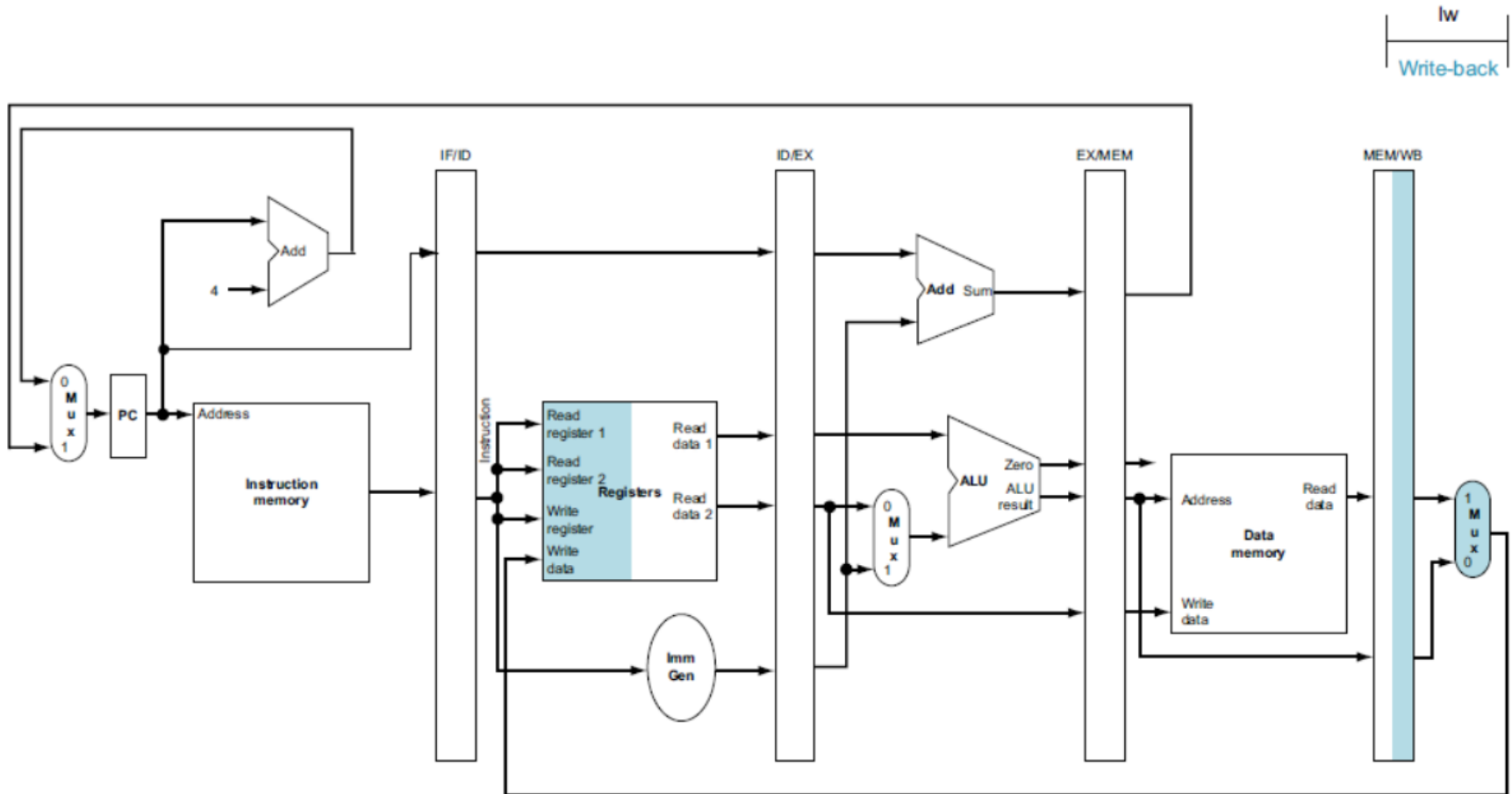


MEM for Load

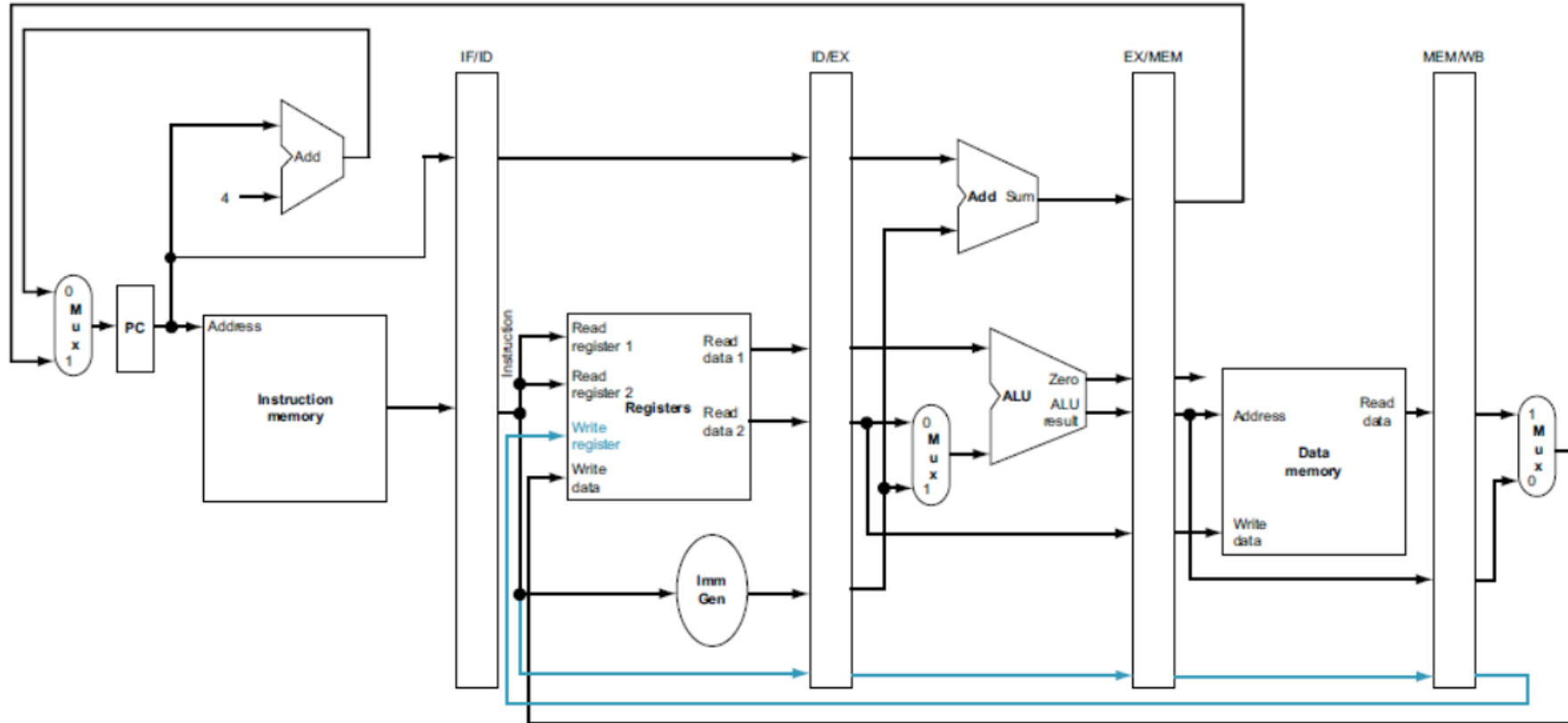


WB for Load

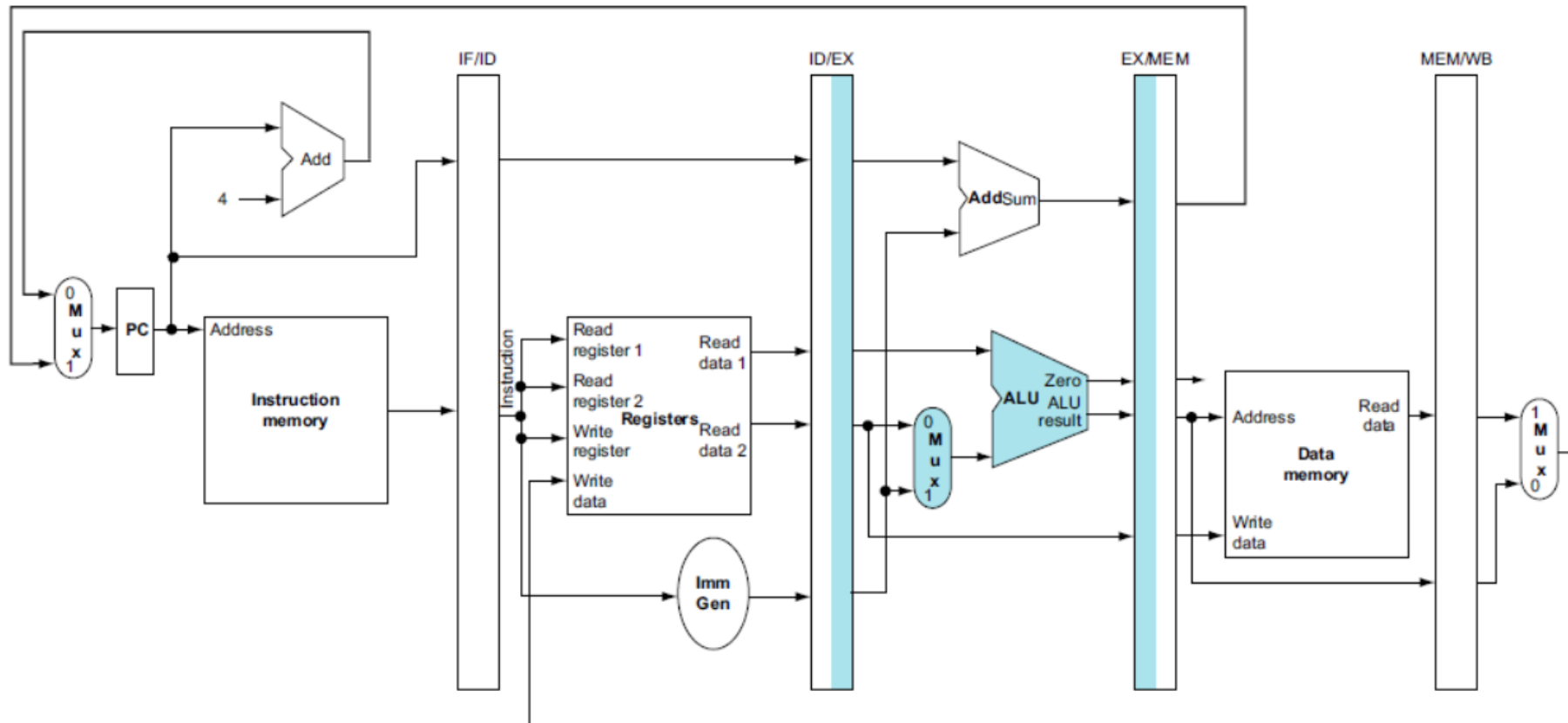
A bug in this figure!



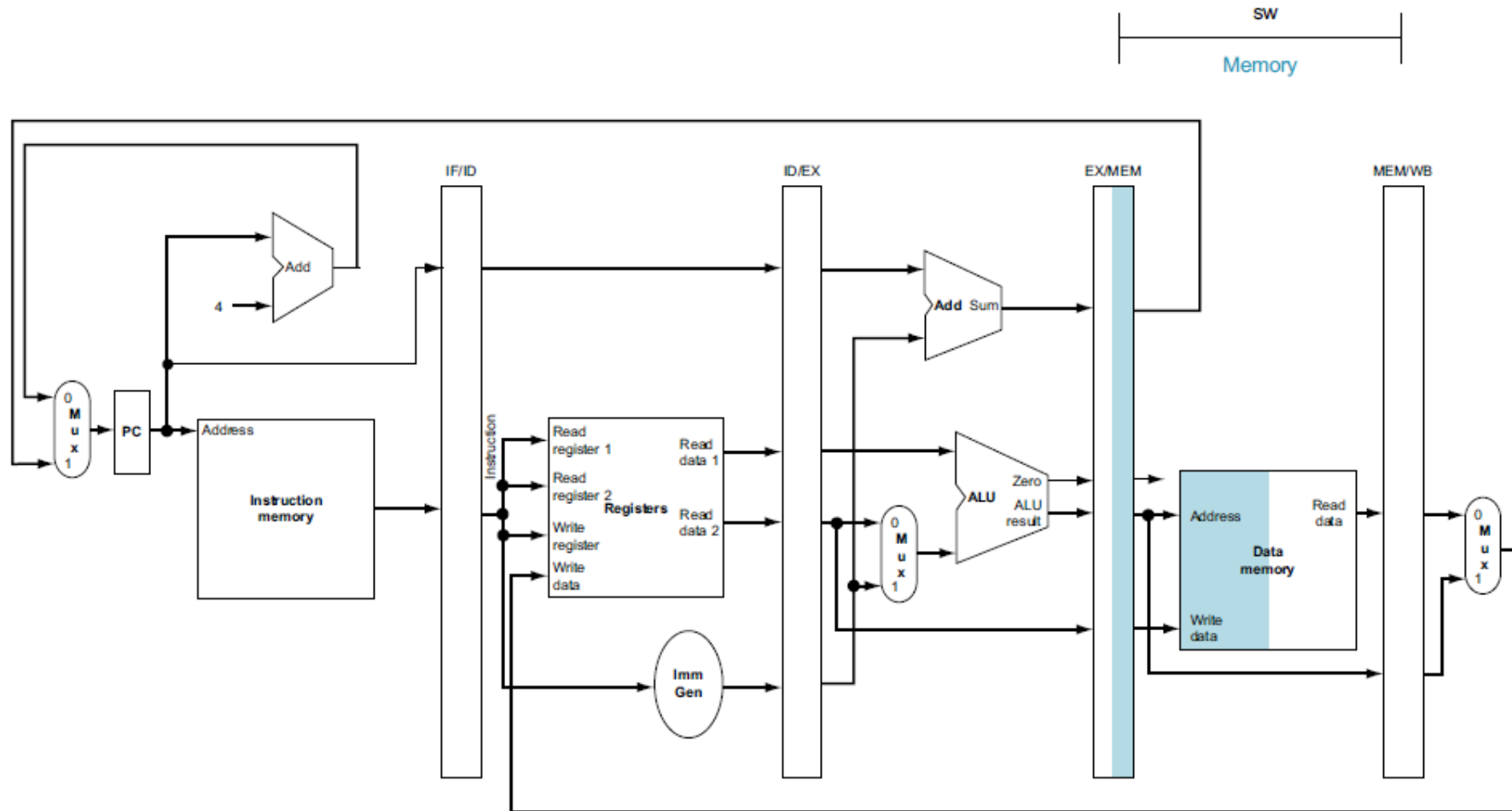
Corrected Datapath for Load



EX for Store

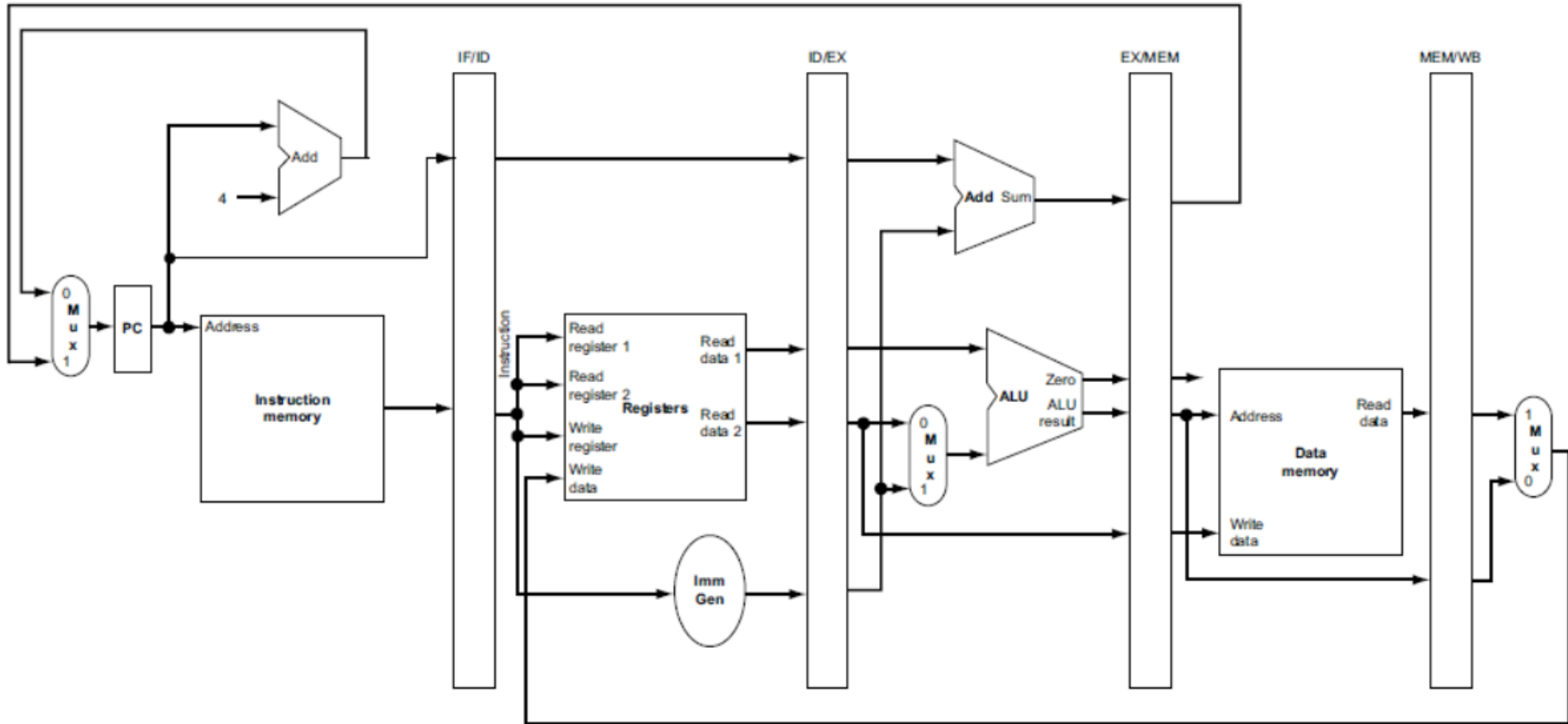


MEM for Store

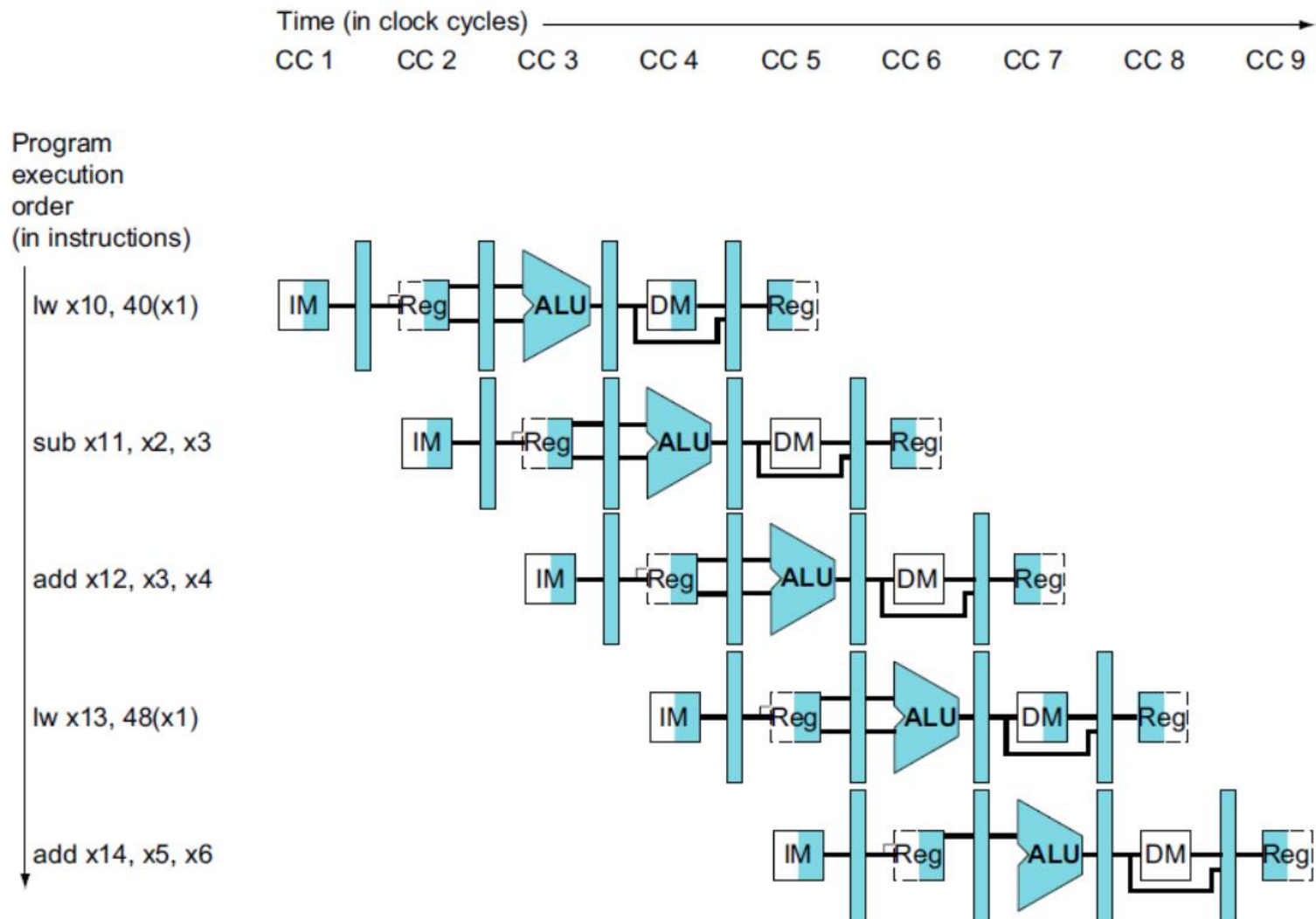


WB for Store

SW
Write-back

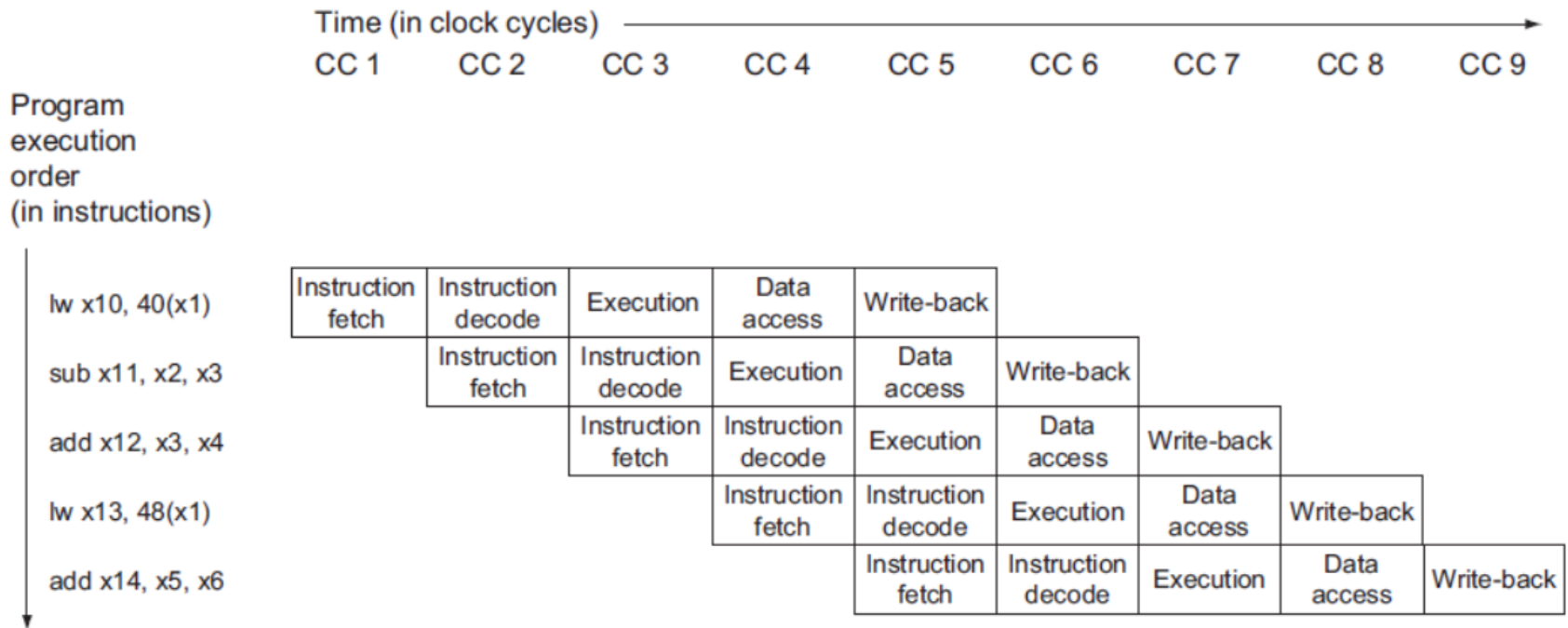


Multi-Cycle Pipeline Diagram



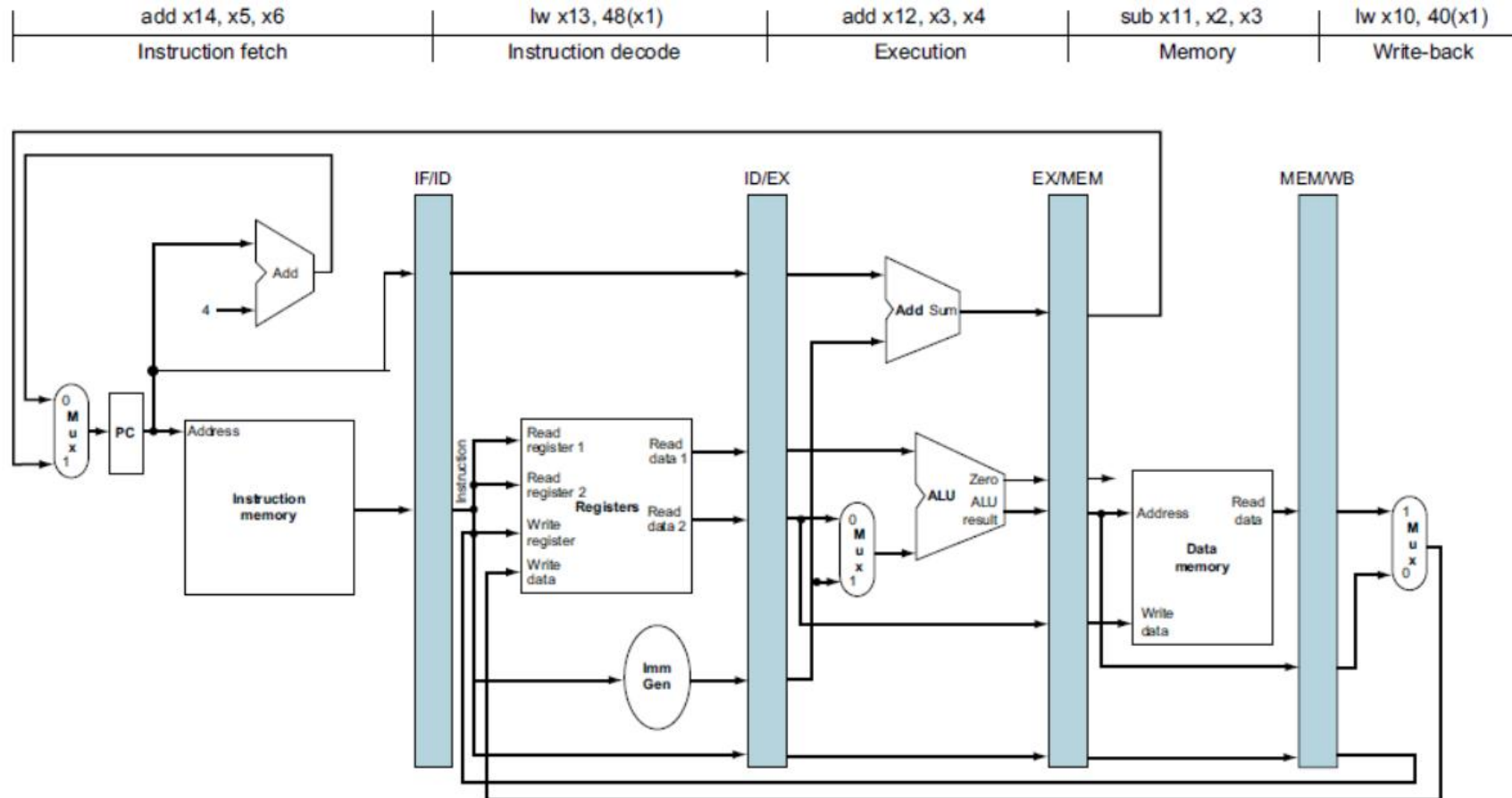
Multi-Cycle Pipeline Diagram

■ Traditional form

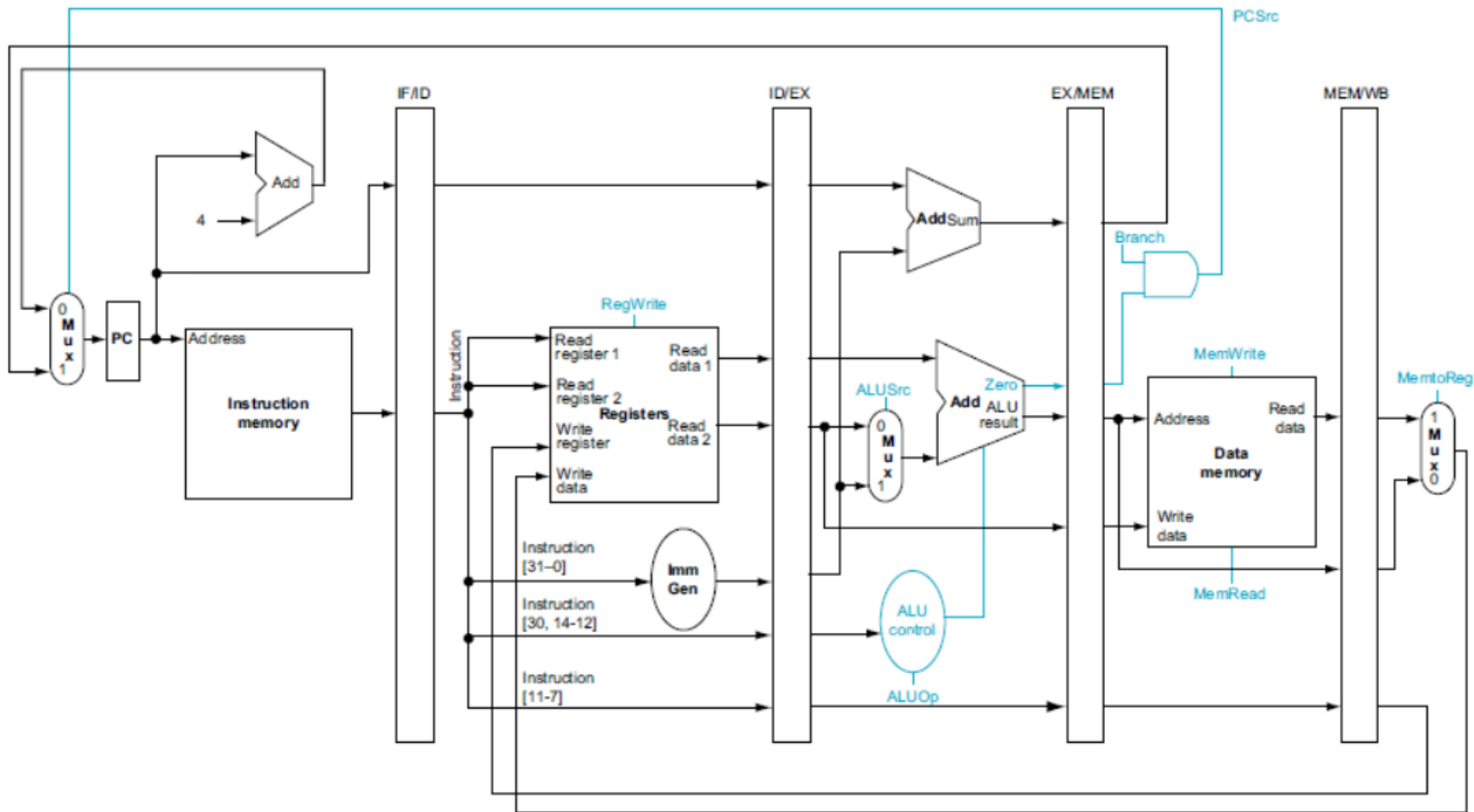


Single-Cycle Pipeline Diagram

- A single-clock-cycle figure is a vertical slice through a multiple-clock-cycle diagram



Pipelined Control (Simplified)



Pipelined Control

- Recall from Single-cycle control

Instruction	ALUOp	operation	Funct7 field	Funct3 field	Desired ALU action	ALU control Input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

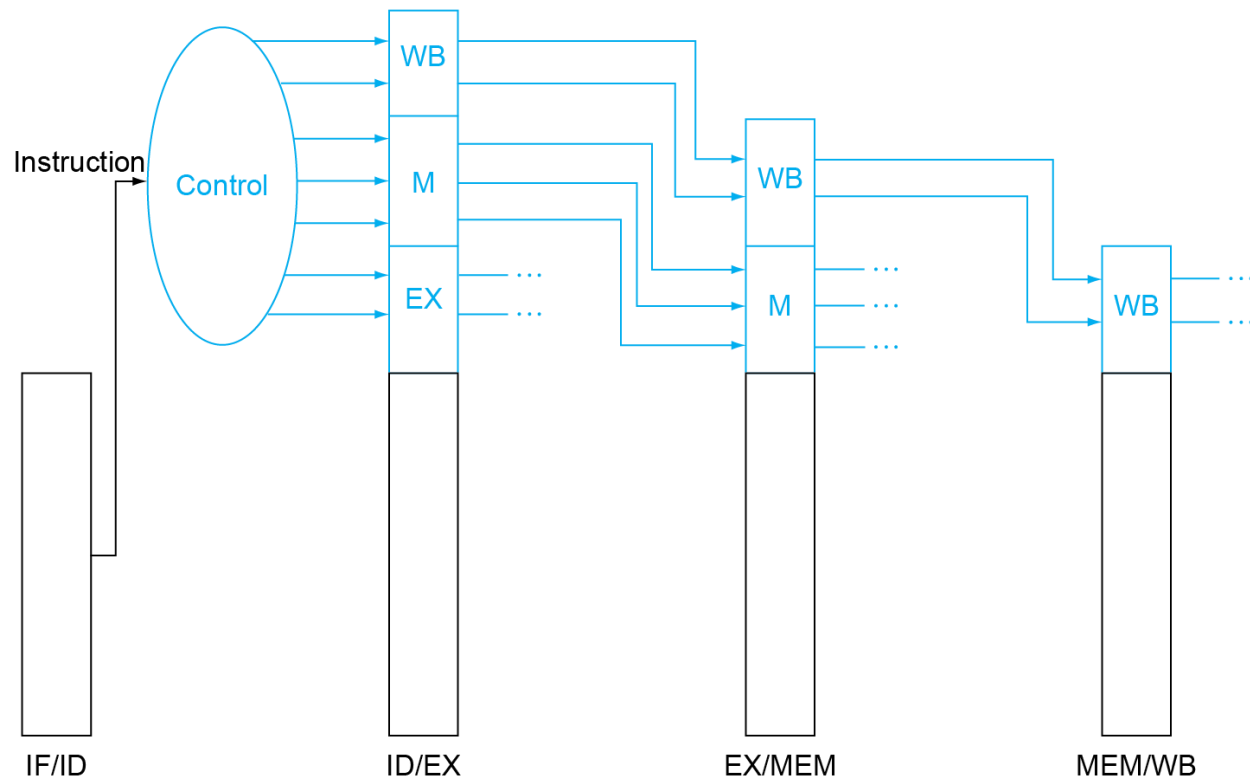
Pipelined Control

- Recall from Single-cycle control

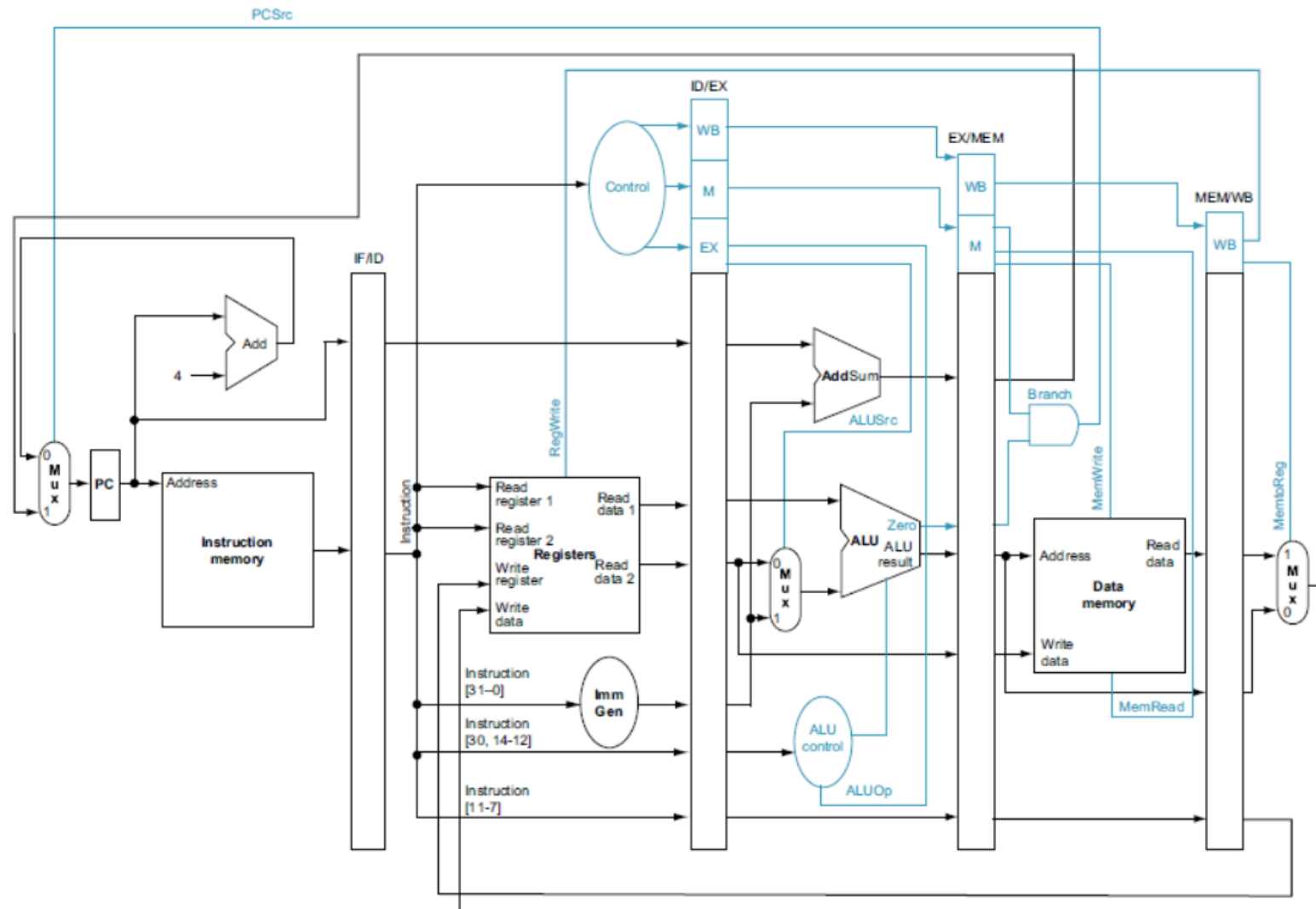
Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

Pipelined Control

- Control signals derived from instruction
 - As in single-cycle implementation



Pipelined Control



Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)